NumPy

Tính trên cả mảng thay cho vòng lặp

Chương 0 · Bài 0.2

NumPy để làm gì

Một phép tính cho cả mảng, không cần vòng lặp, nhanh hơn Python nhiều lần.

Ba khái niệm cần nắm: **shape**, **axis**, **broadcasting**.

Bảng điểm 5 học sinh × 3 môn, shape `(5, 3)`:

- `axis=0`: 3 số, trung bình từng môn.
- `axis=1`: 5 số, trung bình từng học sinh.

Mục tiêu

- Thay vòng lặp bằng phép tính trên cả mảng.
- Đoán shape kết quả khi tính theo axis và khi broadcasting.
- Phân biệt view và copy.
- Tránh các lỗi hay gặp: trừ mảng với , tràn số `uint8`.

Mỗi thuộc tính trả lời một câu hỏi

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]],
             dtype=np.int64)
print(a.shape, a.ndim, a.size, a.dtype)
print(np.arange(12).reshape(3, -1).shape)
```

KẾT QUẢ

```
(2, 3) 2 6 int64
(3, 4)
```

- `shape`: độ dài mỗi trục.
- `ndim`: số trục.
- `size`: tổng số ô.
- `dtype`: kiểu mỗi ô.

`reshape` giữ số ô; chỉ một trục được để `-1`.

Một ảnh là một mảng số

`(427, 640, 3)`



Ảnh màu là mảng (cao, rộng, 3): mỗi điểm ảnh là ba số R, G, B.

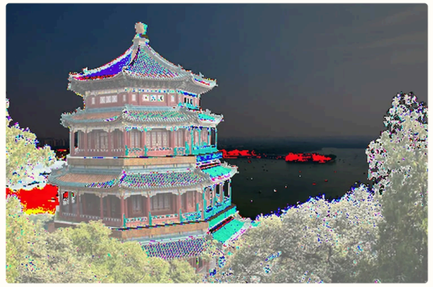
Mười dòng NumPy trên một ảnh



`img[:, ::-1]` → (320, 480, 3): đảo thứ tự cột: lật ngang



`img[::8, ::8]` → (40, 60, 3): lấy mỗi 8 hàng, 8 cột một điểm



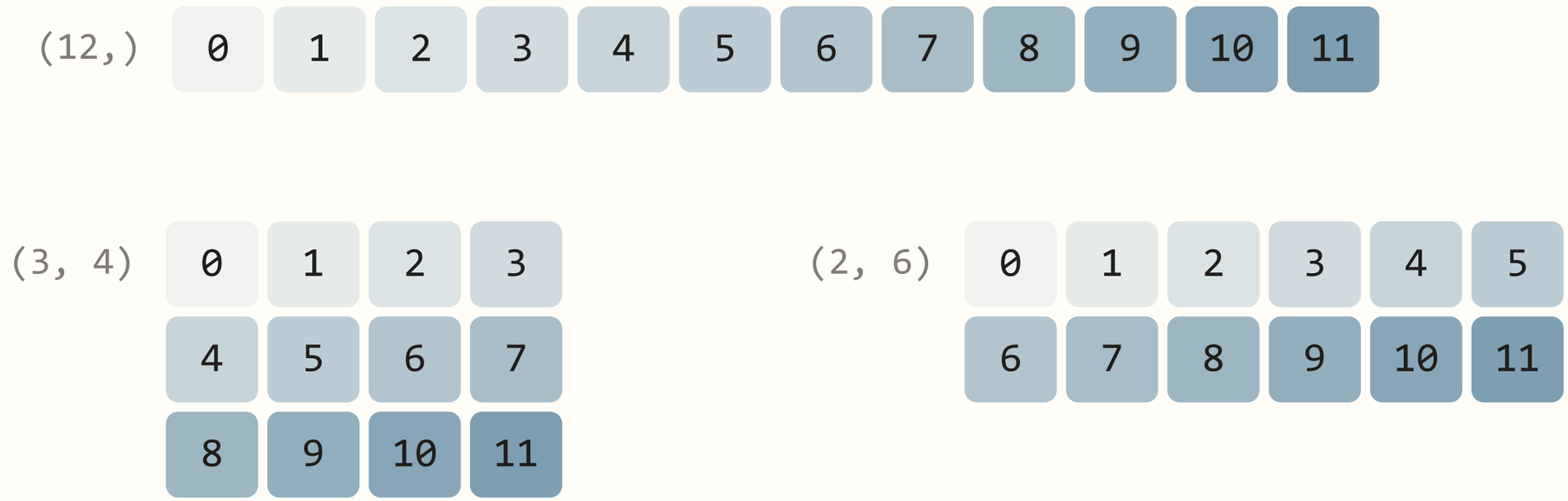
`img + 100` → (320, 480, 3), uint8 tràn vòng: 200 + 100 trong uint8 thành 44: màu sáng bị vỡ



`np.clip(img.astype(int) + 100, 0, 255)` → (320, 480, 3): đổi sang int rồi kẹp về 0–255

Một ảnh thật (320, 480, 3) qua mười dòng NumPy: lát cắt, chọn kênh, gộp trục, broadcast, và cái bẫy tràn số uint8.

reshape chỉ đổi cách đọc



Cùng 12 số, cùng thứ tự; reshape chỉ đổi cách chia hàng.

Số nguyên NumPy có độ rộng cố định

```
pixels = np.array([200, 100], dtype=np.uint8)
print(pixels + 100)
```

KẾT QUẢ

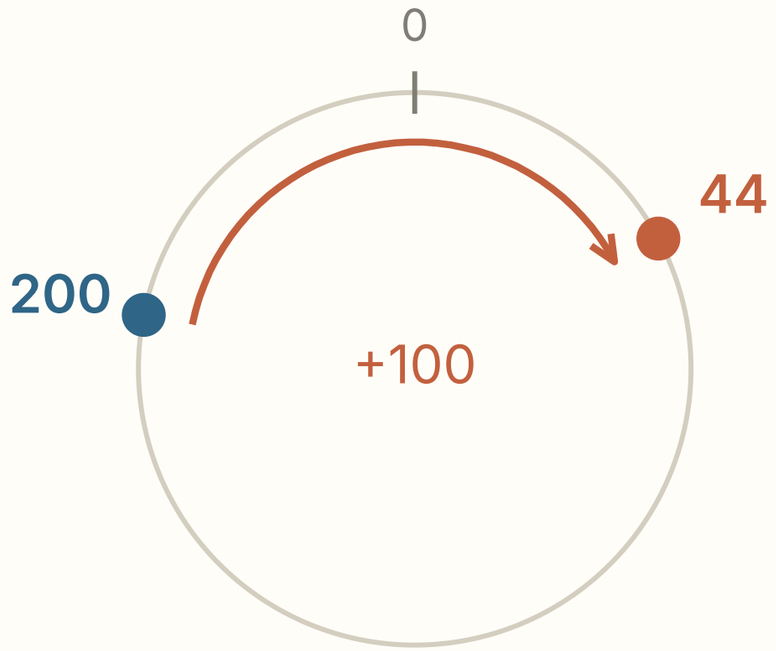
```
[ 44 200]
```

vượt miền 0–255, quay vòng thành 44.

Đổi sang `float32` **trước khi tính**, rồi `np.clip` về 0–255.

Dtype	Byte mỗi ô
`uint8`	1
`float32`	4
`float64`	8

uint8 chạy trên một vòng



uint8 chạy trên một vòng 256 giá trị: 200 + 100 thành 44.

HỎI NHANH

`np.array([250, 10], dtype=np.uint8) + 10` cho ra gì?

A `[260, 20]`

B `[4, 20]`

C `[255, 20]`

D Báo lỗi tràn số

Phép so sánh cũng cho một mảng

```
z = np.array([-2.0, 0.0, 1.0, 3.0])
print(z > 0, (z > 0).sum(), (z > 0).mean())
print(np.where(z > 0, z, 0.0))
```

KẾT QUẢ

```
[False False  True  True] 2 0.5
[0. 0. 1. 3.]
```

`True` tính như 1 khi gộp.

- `sum`: số phần tử thỏa điều kiện.
- `mean`: tỉ lệ thỏa điều kiện.
- Accuracy: `(pred == y).mean()` khi hai shape trùng nhau.

HỎI NHANH

Với `z = np.array([-2.0, 0.0, 1.0, 3.0])`, dòng `if z > 0:` `print("dương")` làm gì?

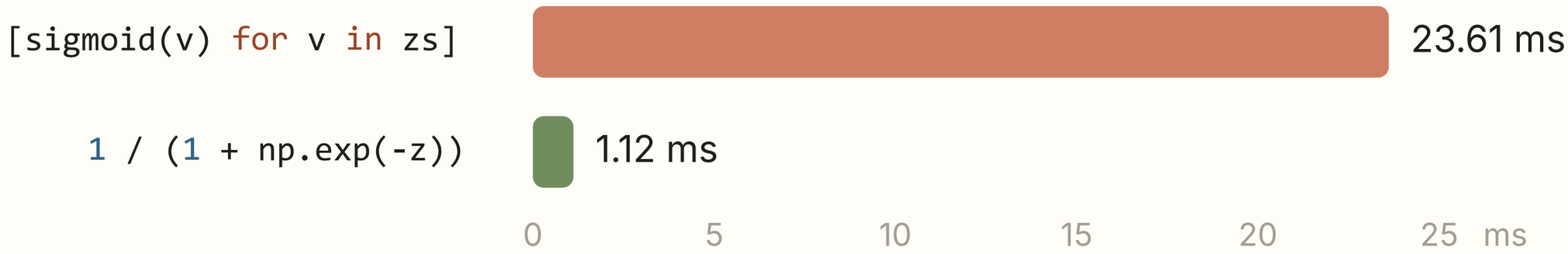
A In “dương”: có phần tử dương

B Không in: không phải mọi phần tử dương

C `ValueError`: không rõ `any` hay `all`

D In “dương” hai lần, mỗi số dương một lần

Cùng công thức, hai cách thực hiện



Sigmoid của 200 000 số, đo trên một máy CPU.

Kiểm hai cách ra cùng kết quả rồi mới đo.

Bốn cách lấy dữ liệu

`a[1, 2]`

0	1	2	3
4	5	**6**	7
8	9	10	11

`a[:, 0]`

0	1	2	3
4	5	6	7
8	9	10	11

`a[-1, ::2]`

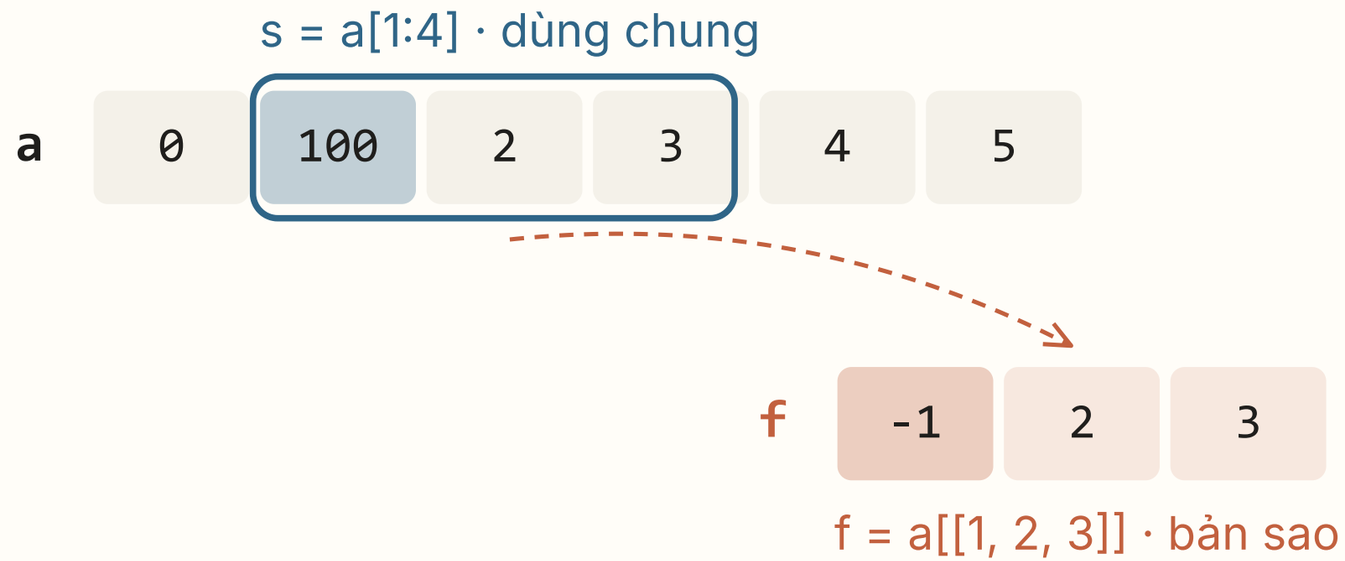
0	1	2	3
4	5	6	7
8	9	**10**	11

`a[a % 5 == 0]`

0	1	2	3
4	**5**	6	7
8	9	**10**	11

Bốn cách lấy dữ liệu từ cùng một mảng 3 × 4.

View và copy



Lát cắt là view của cùng vùng nhớ; fancy indexing chép ra bản sao.

Lấy ra `b = a[a > 5]`: bản sao. Gán thẳng `a[a > 5] = 0`: sửa `a`.

HỎI NHANH

Chạy `a = np.zeros(5)`, `b = a[2:]`, `b += 1`. Lúc này `a` là gì?

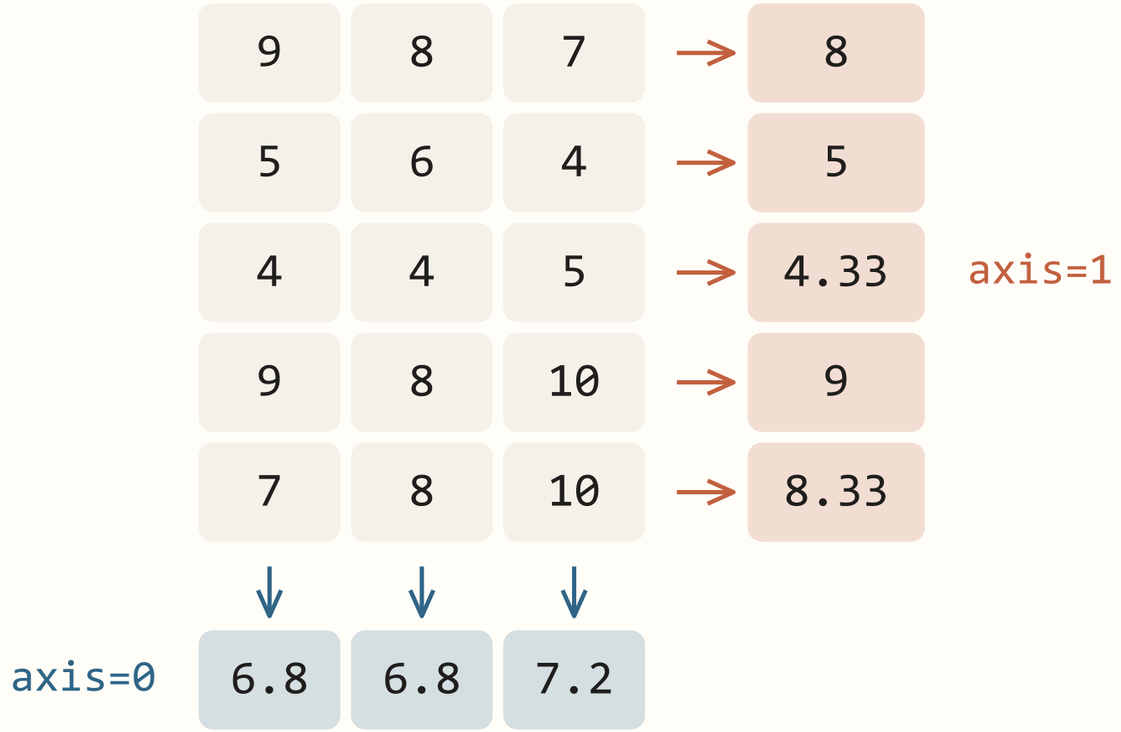
A `[0, 0, 0, 0, 0]`

B `[1, 1, 1, 1, 1]`

C `[0, 0, 1]`

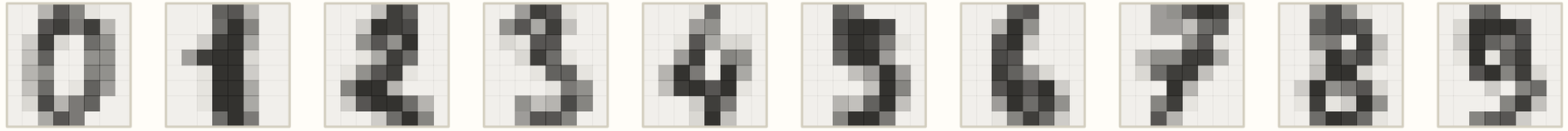
D `[0, 0, 1, 1, 1]`

Trục nào được gộp, trục đó biến mất



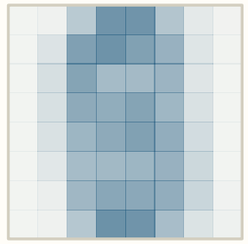
Trục ghi trong axis là trục bị gộp và biến mất.

Gộp theo axis trên ảnh thật



axis=(1, 2)	4.6	4.9	5.4	4.2	4.0	5.3	4.8	4.5	5.6	5.1

axis=0



1797 chữ số viết tay 8 × 8: độ sáng từng ảnh và ảnh trung bình.

Keepdims giữ ý nghĩa trục

Giả sử `imgs` là batch ảnh xám có shape `(100, 28, 28)`.

```
print(imgs.mean(axis=0).shape)
print(imgs.mean(axis=(1, 2)).shape)
print(imgs.mean(axis=(1, 2),
                keepdims=True).shape)
```

KẾT QUẢ

```
(28, 28)
(100,)
(100, 1, 1)
```

- Gộp ảnh: một ảnh trung bình.
- Gộp không gian: một độ sáng mỗi ảnh.
- `keepdims=True`: các trục gộp thành độ dài 1.

HỎI NHANH

`X` có shape `(32, 3, 64, 64)`, tức 32 ảnh màu, 3 kênh, 64 × 64 điểm ảnh. `X.mean(axis=(0, 2, 3))` có shape gì?

A `(3,)`

B `(32,)`

C `(64, 64)`

D `(32, 3)`

Nối trục cũ, hay tạo trục mới?

Tiếp từ bảng `scores` có shape `(5, 3)` ở phần `axis`.

```
joined = np.concatenate(
    [scores[:2], scores[2:]], axis=0)
two_sessions = np.stack(
    [scores, scores], axis=0)
print(joined.shape,
      np.array_equal(joined, scores))
print(two_sessions.shape)
```

KẾT QUẢ

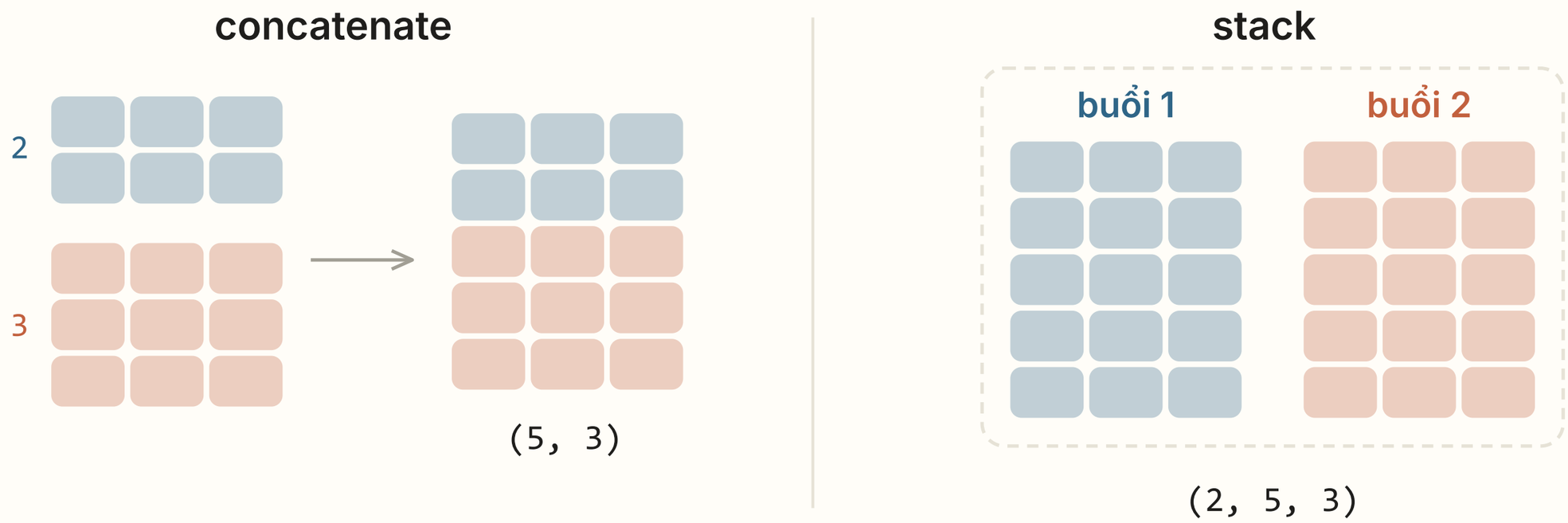
```
(5, 3) True
(2, 5, 3)
```

`concatenate`: nối trên trục đã có.

`stack`: thêm trục buổi đo.

Mỗi trục có ý nghĩa riêng: `(buổi, học sinh, môn)`.

concatenate và stack



concatenate nối trục đã có; stack thêm một trục mới.

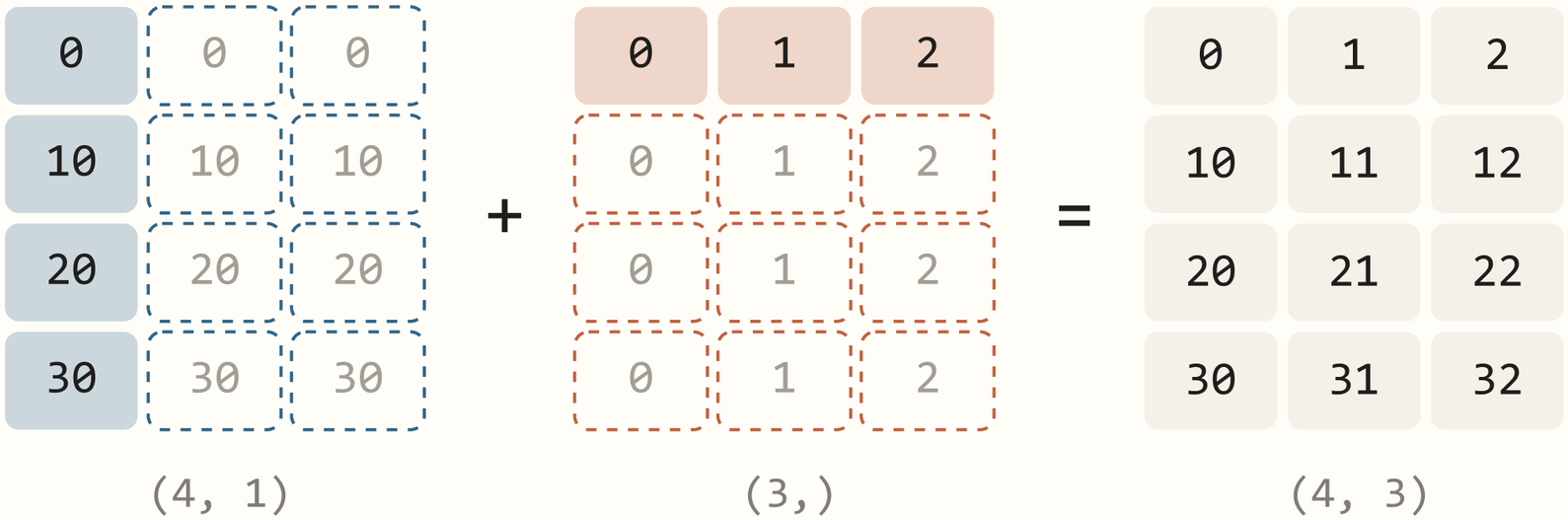
Xếp từ phải, so từng trục

1. Thiếu trục thì coi như thêm độ dài 1 bên trái.
2. Mỗi trục: bằng nhau hoặc một bên bằng 1.

A	B	Kết quả
`(5, 3)`	`(3,)`	`(5, 3)`
`(5, 3)`	`(5, 1)`	`(5, 3)`
`(5, 3)`	`(5,)`	Lỗi

Mảng nhỏ không bị chép thật, nhưng kết quả vẫn đủ kích thước.

Một cột cộng một hàng



Trục độ dài 1 được kéo dãn cho khớp; ô nét đứt không bị chép thật.

Chuẩn hóa theo từng môn

Tiếp từ `scores.shape == (5, 3)`; mỗi cột là một môn.

```
mu = scores.mean(axis=0)
sigma = scores.std(axis=0)
sigma_safe = np.where(sigma == 0, 1.0, sigma)
Z = (scores - mu) / sigma_safe
```

- `mu`, `sigma`: `(3,)`, một số mỗi môn.
- Cột hằng: mẫu số thay bằng 1, kết quả toàn 0.
- Khi làm model: tính `mu`, `sigma` trên train, dùng lại cho test (bài 1.4).

Khoảng cách bốn cặp

, .

```
A = np.array([[0.0, 0.0], [3.0, 4.0]])
B = np.array([[0.0, 4.0], [3.0, 0.0]])
diff = A[:, None, :] - B[None, :, :]
D = np.sqrt((diff ** 2).sum(axis=-1))
print(D)
```

KẾT QUẢ

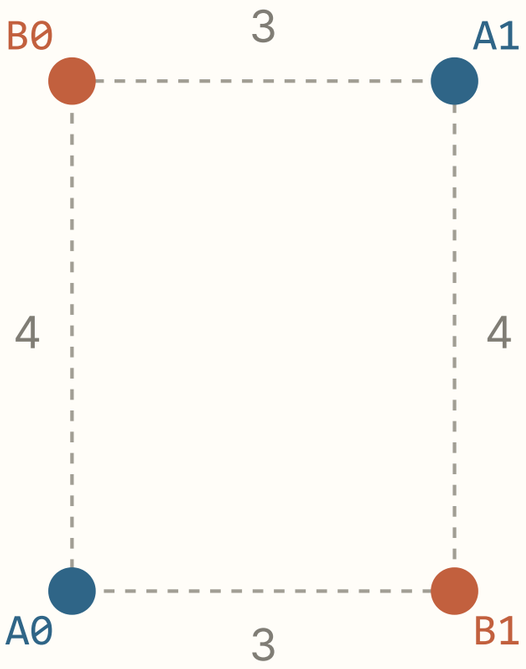
```
[[4. 3.]
 [3. 4.]]
```

`diff`: `(2, 2, 2)`; gộp trục tọa độ để còn `(2, 2)`.

Mỗi ô là một cặp, không phải một tọa độ.

Mảng lớn cần ô: số float64 dùng **51.2 GB**, chưa gồm mảng tạm khác.

Bốn độ dài trên hình, bốn ô trong D



	B0	B1
A0	4	3
A1	3	4

D

Ô (i, j) của D là khoảng cách từ Ai tới Bj.

HỎI NHANH

`A` có shape `(8, 1, 6)`, `B` có shape `(7, 1)`. `A + B` có shape gì?

A Báo lỗi, vì khác số trục

B `(8, 7, 1)`

C `(8, 1, 6)`

D `(8, 7, 6)`

Dự đoán đúng hết, MSE vẫn sai?

```
y = np.array([1.0, 2.0, 3.0])
y_hat = np.array([[1.0], [2.0], [3.0]])
print((y_hat - y).shape)
```

KẾT QUẢ

```
(3, 3)
```

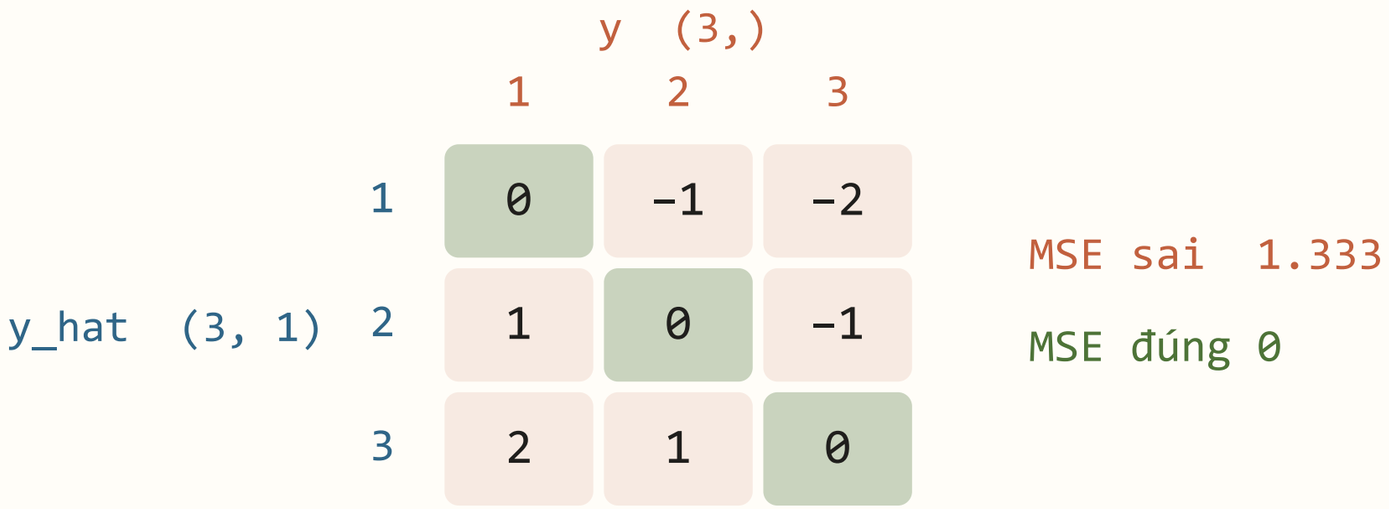
MSE ra thay vì .

Broadcasting tạo mọi hiệu .

Sửa: đưa `y_hat` về `(n,)` bằng `.ravel()`.

Phòng: `assert y_hat.shape == y.shape` trong hàm tính loss.

Chín hiệu thay vì ba phần dư



(3, 1) − (3,) thành bảng 3 × 3; chỉ đường chéo là phần dư thật.

HỎI NHANH

`y_hat` có shape `(1000, 1)`, `y` có shape `(1000,)`. Ta tính `np.mean((y_hat - y) ** 2)`. Kết quả ra sao?

A Ra một số sai, không báo lỗi

B Báo lỗi, vì hai shape khác nhau

C Ra đúng MSE, vì NumPy tự khớp shape

D Chỉ tính trên mẫu đầu tiên

Nhân ma trận: @, không phải *

```
M = np.array([[2.0, 1.0], [1.0, 3.0]])
v = np.array([1.0, 2.0])
print(M @ v, M * v)
```

KẾT QUẢ

```
[4. 7.] [[2. 2.]
 [1. 6.]]
```

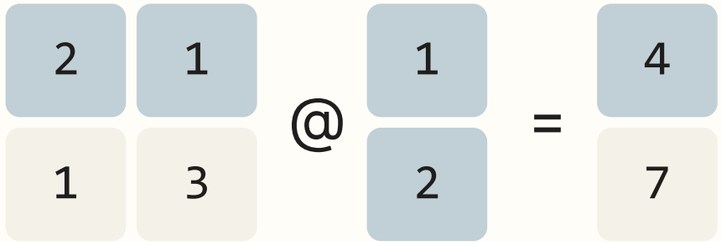
`@`: mỗi hàng nhân vô hướng với v.

`*`: từng ô nhân theo broadcasting.

`np.linalg.solve(M, v)`: giải hệ .

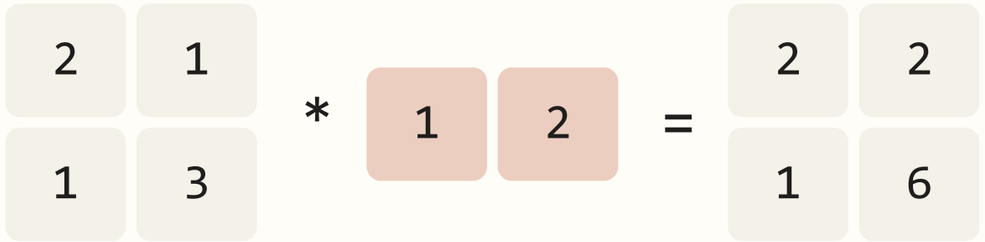
Hàng nhân vector, hay nhân từng ô?

`M @ v`



`2·1 + 1·2 = 4`

`M * v`



`nhân từng ô`

@ là tích vô hướng từng hàng; * nhân từng ô.

HỎI NHANH

Ma trận có 3 hàng, 4 cột. Với mọi , ta có . Hàng đầu của là gì?

A `[1, -1, 0, 0]`

B `[0, 0, 1, 0]`

C `[0, 0, 1, -1]`

D `[1, 2, 0, 0]`

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Problem 3.1

HỎI NHANH

Tạo `rng = np.random.default_rng(42)`, rồi lấy `a = rng.integers(0, 10, size=5)` và `b = rng.integers(0, 10, size=5)`. `a` có bằng `b` không? Chạy lại cả đoạn thì có ra đúng số cũ không?

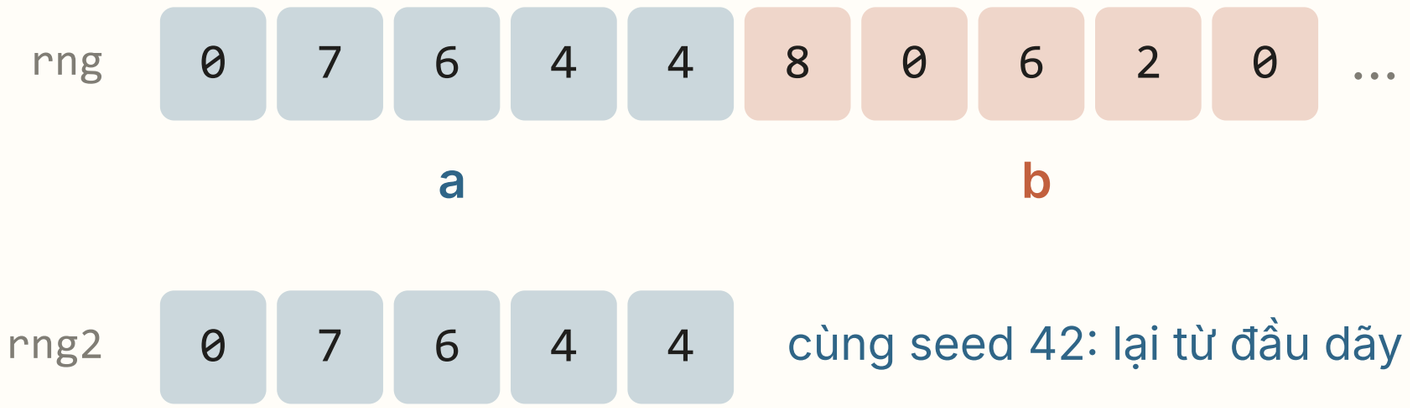
A `a` bằng `b`, và chạy lại ra đúng số cũ

B `a` khác `b`, và chạy lại ra đúng số cũ

C `a` bằng `b`, và chạy lại ra số mới

D `a` khác `b`, và chạy lại ra số mới

Seed cố định cả một dãy



Seed cố định cả dãy; mỗi lần gọi lấy phần tiếp theo.

Sắp xếp và đếm có sẵn

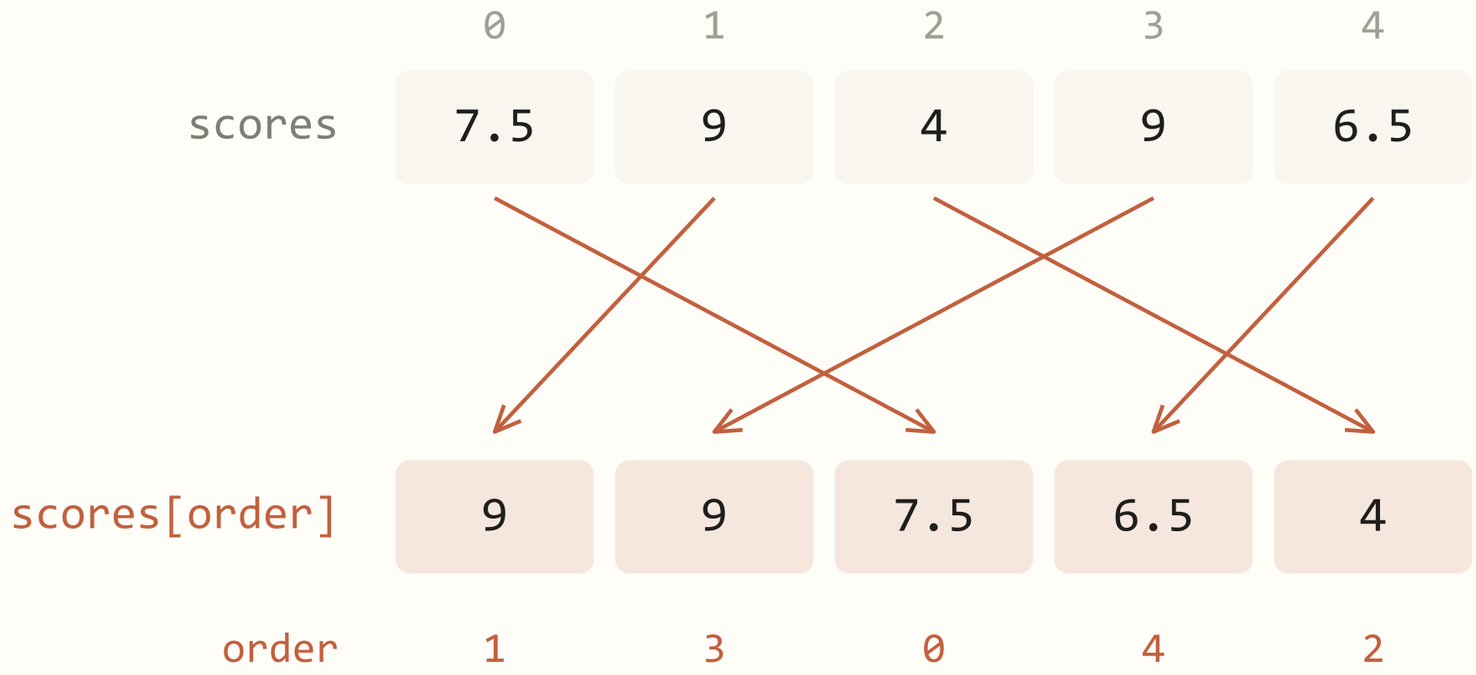
```
scores = np.array([7.5, 9.0, 4.0, 9.0, 6.5])
order = np.argsort(-scores, kind="stable")
print(order, scores[order])
labels = np.array([2, 0, 2, 1, 2, 0])
print(np.bincount(labels, minlength=4))
```

KẾT QUẢ

```
[1 3 0 4 2] [9.  9.  7.5 6.5 4. ]
[2 1 3 0]
```

- `argsort` trả **chỉ số**, không trả giá trị.
- `argpartition`: lấy lớn nhất, như `nth_element`.
- `bincount`, `unique`: đếm nhãn.
- `cumsum`, `searchsorted`: cộng dồn, tìm nhị phân.

Đọc scores theo order



Đọc scores theo order được dãy giảm dần.

HỎI NHANH

`np.argsort(np.array([30, 10, 20]))` in ra gì?

A `[2 0 1]`

B `[1 2 0]`

C `[10 20 30]`

D `[0 1 2]`

Khi đi thi

- In shape sau mỗi bước; `assert` trước khi tính loss.
- `uint8` → `float32` trước khi cộng trừ.
- Tính bộ nhớ mảng trung gian trước khi broadcast.

02

Luyện đề

LUYỆN ĐỀ 1

Cho `A = [[1, 2], [3, 4]]` và `B = [[0, 1], [1, 0]]` là hai mảng NumPy. `A * B` và `A @ B` lần lượt bằng gì?

A Cả hai đều bằng `[[2, 1], [4, 3]]`

B `[[0, 2], [3, 0]]` và `[[2, 1], [4, 3]]`

C `[[2, 1], [4, 3]]` và `[[0, 2], [3, 0]]`

D `[[0, 2], [3, 0]]` và `[[3, 4], [1, 2]]`

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 113

LUYỆN ĐỀ 2

`X` có shape `(1000, 20)`: 1000 mẫu, 20 đặc trưng. Muốn chuẩn hóa riêng từng đặc trưng, ta viết thế nào?

A
```
(X - X.mean(axis=1)) / X.std(axis=1)
```

B
```
(X - X.mean(axis=0)) / X.std(axis=0)
```

C
```
(X - X.mean()) / X.std()
```

D
```
X / X.max()
```

LUYỆN ĐỀ 3

`A` và `B` đều có shape `(20000, 128)`, kiểu `float64`. Tính `A[:, None, :] - B[None, :, :]` thì mảng trung gian tốn khoảng bao nhiêu bộ nhớ?

A 400 GB

B 20 MB

C 400 MB

D 4 TB

Chốt ý

Trục ghi trong `axis` thì biến mất.

Broadcasting xếp shape từ phải; cẩn thận `(n,)` với `(n, 1)`.

`@` nhân ma trận, `*` nhân từng ô.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook: đoán shape của mỗi dòng trước khi chạy, rồi kiểm lại.